YouTube Video Summarizer

Project Submission

Candidate Name: Anmol Gupta
Role: Business Analyst
Date: 20 April 2026

1. Objective

To build a simple tool that takes a **YouTube video link** as input and generates a **clean summary of the video**.

The tool allows the users to quickly understand the key insights and summary of a video without having to watch the entire content for free.

The generated output includes:

- **Short Summary (3–4 lines)**
- **Detailed Summary**
- **Key Bullet Points**
- **Actionable Insights / Key Takeaways**

2. Approach

The primary objective of the project was to design a robust YouTube video summarization system capable of handling both—videos having transcripts and videos not having transcripts.

The tool takes two primary inputs from the user: a YouTube video URL and a preferred output language—either English or the original language of the video.

The project was developed using an AI-assisted workflow, where system design and requirements were defined manually, and implementation was carried out using AI agents. GPT-based models were used for planning and structuring the pipeline, while the Opal agent from Gemini Pro was used to generate and refine the implementation.

An adaptive multi-stage approach was implemented to ensure that if transcripts are unavailable, the system falls back to speech-to-text, making it functional for all videos. The system supports multilingual output by leveraging the generative capabilities of the LLM rather than relying on separate translation services.

The output is generated in a dynamic user-friendly HTML webpage making it easier for users to interpret and copy the output making it easier to share and analyse it further. Additionally, the output also gives the tone of the video, enabling users to better interpret the video.

3. System Workflow

The workflow of the system can be summarized as follows:

User input (YouTube URL + Language)
↓
Transcript extraction
↓
(If unavailable → Speech-to-text)
↓
Text processing and Chunking
↓
AI Summarization
↓
Final Output

4. Edge Cases Considered

The following edge cases were considered while designing the system to ensure the tool works reliably under multiple conditions and across different types of inputs.

1. Videos with No Transcript Available

The system addresses the absence of transcripts by implementing a fallback mechanism using speech-to-text technology. If captions are unavailable, the video audio is automatically extracted and converted into text. This ensures that pipeline remains functional for all videos, regardless of transcript availability

---

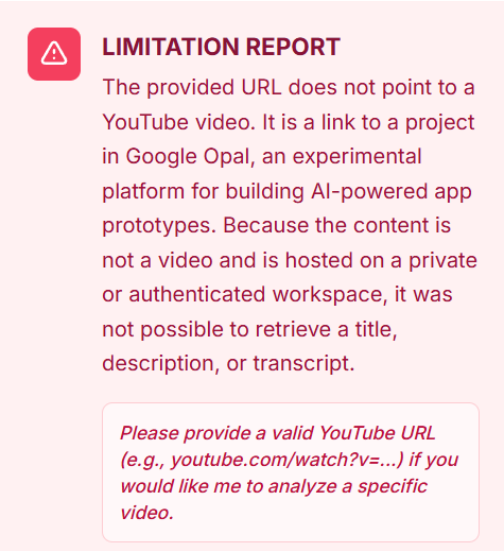
2. Very Long Videos Where Transcript Size May Exceed Model Limits

The system handles long videos by splitting the transcript into smaller chunks and summarizing each part separately. These smaller summaries are then combined into a final summary. This approach avoids model input limits while maintaining overall context and coherence.

---

3. Invalid or Private YouTube Links

If the user enters an invalid YouTube link or a link to a private or restricted video, the system may not be able to access the content. In such cases, the system prompts the user to provide a valid public YouTube URL.



---

4. Non-English Videos

The system handles non-English videos by generating summaries directly in the language input by the user. Instead of relying on separate translation tools, the model is guided to understand and summarize the content in English or the original language of the video. This ensures multilingual output within a single pipeline.

---

5. Short-Form Videos with Very Little Transcript

Short videos contain limited content with very little transcript. The system still tries to analyse the video, even if the transcript is not available at all, so that it can generate actional insights.

5. Challenges Faced

During the development of this prototype, the following practical challenges were faced and tackled:

1. Controlling LLM Output Consistency

LLMs can sometimes generate summaries that are inconsistent or longer than necessary. Prompt engineering was used to enforce structure (e.g., concise summaries, bullet points) and

maintain consistent output quality.

---

2. AI-Assisted Development Challenges

Relying on AI agents for development can lead to incorrect or inconsistent code generation. An iterative prompt refinement strategy was followed, where outputs were reviewed and improved step-by-step to ensure correctness. The prompts were edited multiple times in each step so that the model behaves exactly as required.

---

3. Choosing and Integrating Multiple AI Tools Smoothly

Multiple AI tools were used keeping in mind that they’re free to use, reliable and don’t require investment over use. Combining different tools and libraries can lead to compatibility and workflow issues. A modular integration approach was used, ensuring each tool operates independently while contributing to the overall pipeline.